

Machine learning in production

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 1, `nw/triage`: a calibrated four-class priority classifier behind a FastAPI service. Liveness and readiness, Prometheus metrics, a correlation ID, a versioned artifact with a checksum, and a container with the model baked in.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Project 1 orders Northwind's queue by priority. The agent calls it as a tool; the capstone puts it in front of the agent as a cheap pre-filter. It has to be right about P0 far more often than about anything else.

Today, step by step

- 1 Look at the data and the feature pipeline
- 2 Train the model as given and read the confusion matrix
- 3 Class weights
- 4 A threshold, not argmax
- 5 Calibrate the probabilities
- 6 Alive is not ready
- 7 Build the container
- 8 Session check and commit
- 9 Argue with the threshold **OPTIONAL**

Read the P0 row first

Steps 1 and 2 train a model that already passes most of the suite. Do not fix it until you have read what it gets wrong.

At the top of step 6, start `make image-triage` in a second terminal. The build takes minutes and step 7 needs it.

Accuracy is a trap on four percent P0

4%

of Northwind's tickets are P0. A model that never predicts P0 is 96 percent accurate on that class and useless

Test split	Accuracy	Macro - F1	P0 recall
Always predict P2	0.41	0.15	0.00
The model as shipped	0.69	0.69	0.58
What Northwind needs	lower	higher	at or above 0.8

Pick the metric before you train, and pick one the business can argue about.

The naive model sends 13 real emergencies to the ordinary queue

18 of 31

true P0 tickets caught by argmax on the test split. The P0 row reads 18, 13, 0, 0

- 31 real P0s in the test split; 13 predicted P1, where they wait behind ordinary work
- `make session02` at this point: two failures and one error
- Everything else already passes, which is the point

A model can pass most of a test suite and still be useless for the one thing the business needs. The failing test is named for it: `test_p0_recall_is_protected_by_the_threshold`.

Five metrics, and only one is the commercial number

Metric	What it answers	Why it matters here
Accuracy	How often is the top guess right?	Dominated by the common classes; hides P0
Macro - F1	Average F1 across four classes, equally weighted	Rewards getting the rare class right
P0 recall	Of true P0s, how many were flagged?	The commercial number: a missed P0 is an SLA breach
P0 precision	Of flagged P0s, how many were real?	The cost of the recall you buy
Brier and ECE	Do the probabilities mean what they say?	A threshold on a bad probability is a threshold on noise

Three fixes for imbalance, in this order

1. Class weights

Charge a P0 mistake by the inverse of its frequency, about ten times a P2 mistake.

Moves the boundary.

Probabilities stay biased.

2. A threshold

Predict P0 whenever its probability clears a value chosen on validation to hit a recall target.

The business decision lives here.

3. Calibration

Map scores to real probabilities on held-out data, so 0.3 means three in ten are real.

Makes the threshold honest.

Resampling is a fourth option. On short text with a linear model it rarely beats class weights and it makes the validation split lie, so it is not in the acceptance criteria.

Weights move the boundary; they do not let you choose where it sits

```
clf = LogisticRegression(  
    max_iter=2000, C=C,  
    class_weight=class_weight,  
)
```

One argument in `build_pipeline`. Retrain and compare.

Report line	Before	After
Macro-F1	0.69	0.72
P0 recall, argmax	0.58	0.77
P0 caught	18 of 31	24 of 31

Still not the level the business needs.

The notebook shows the two confusion matrices side by side. Look at where the seven remaining P0 misses went.

The threshold is a business decision; the code enforces it

Run	Threshold	P0 recall	P0 precision
Default: target 0.90, floor 0.25	0.05	about 0.90	under 0.5
<code>--min-precision 0.5</code>	0.09	0.90	0.56
<code>--min-precision 0.6</code>	0.17	0.84	0.67
<code>--target-recall 0.97</code>	0.48	0.68	0.91

- Sweep 0.05 to 0.95 on the validation split; the lowest threshold that hits the target with precision above the floor
- No threshold does both: fall back to the best P0 F1, silently. The 0.97 row is that fallback
- Macro-F1 falls from 0.72 to 0.64 once the rule fires: recall paid for in P1 tickets flagged P0

Never on the test split. The test split is the number you report. Choosing on it makes that number a lie.

Calibration is measured, not assumed

0.09

ECE after sigmoid calibration on 795 validation rows.
Uncalibrated: 0.058

- Freeze the fitted classifier, wrap it in `CalibratedClassifierCV`, fit the wrapper on the validation split's features
- Brier (P0) 0.012 to 0.014, ECE 0.058 to 0.09: slightly worse
- A linear model that was already close to calibrated buys nothing here. The point is that you measured it

The notebook's calibration cell prints predicted confidence against actual accuracy per bucket, so you can see which buckets the 0.09 comes from.

One fitted object, one file, one checksum

The pipeline in `nw/triage/features.py`: word and character TF-IDF over subject and body, plus eight structural signals such as the share of capital letters and the number of pasted log lines.

Every transform is a fitted scikit-learn object and serialises with the model, so the service cannot tokenise differently from training. The first feature test proves it: same ticket, same vector, before and after a reload.

Why linear today. Logistic regression over n-grams trains in seconds, its coefficients can be read, and on short support text it is hard to beat by more than a few points with anything heavier. The next project measures exactly what heavier buys.

The artifact

`model.joblib`, `metadata.json` (version, git SHA, data hash, threshold, classes, checksum, metrics), `report.json`. `latest` is a symlink, not a copy.

A checksum mismatch is refused at load: corrupt or mixed, either way it does not serve.

Five endpoints, five different callers

Endpoint	Answers	Who calls it
<code>/healthz</code>	Is the process up?	The liveness probe. Failing it restarts the container
<code>/readyz</code>	Can it score a ticket right now?	The load balancer. Failing it stops traffic without a restart
<code>/triage</code>	Prediction, probabilities, the rule that fired, the model version	Project 4's agent, and the capstone router
<code>/metrics</code>	Counts, latency histogram, model version	The scrape target for dashboards and alarms
<code>/version</code>	The artifact's metadata, minus the metrics	Whoever asks "which model is this"

A model that fails to load leaves the process alive and not ready

Load succeeds

`state.ready = True` , `MODEL_LOADED.set(1)` ,
`MODEL_INFO` labelled with the version, one info line
with the path and the version.

Load fails

Process stays up. `state.model = None` ,
`state.ready = False` , `MODEL_LOADED.set(0)` , one
log line with the path and the error.
`/healthz` 200. `/readyz` and `/trriage` 503 until a
model is loaded.

Crashing loops the restart and scrolls the one log line that says why off the screen.

The model is baked in at build and loaded once

Two stages

A builder installs the locked dependencies; the runtime image gets the virtualenv, the package, the model and one data file.

Non-root, probed, graceful

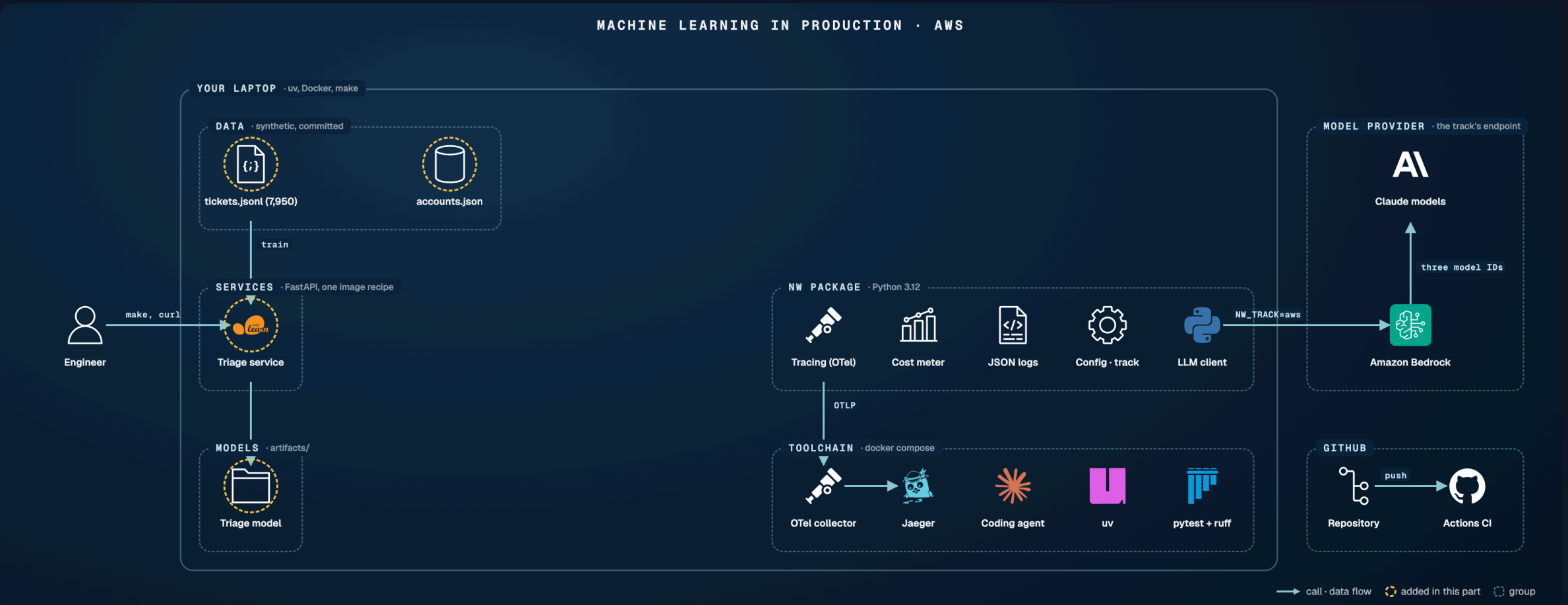
An unprivileged user, a health check on `/readyz`, and a shutdown timeout so in-flight requests finish on a rolling deploy.

Nothing else leaks in

`docker history` shows no layer copying `.env` or the tickets. `.dockerignore` keeps them out; `data/accounts.json` is the only data file.

`docker run --rm -p 8001:8000 nw-triage` answers `/readyz` with the same model version as the local run. Ctrl-C exits within the graceful timeout, not instantly.

What exists after today, AWS track



Your laptop: the committed tickets, the triage service and its artifact, the nw package and the local toolchain. Bedrock stays the track's endpoint; nothing today calls it. Ringed: what this project adds.

What exists after today, Google Cloud track

MACHINE LEARNING IN PRODUCTION · GOOGLE CLOUD



Same service, same artifact, same toolchain; Vertex AI is the endpoint. The triage service does not know which track it is on.

Acceptance criteria

1. Same ticket, same features, before and after reload
2. Macro-F1 clearly above the majority baseline
3. P0 recall at or above 0.8 on the test split, by the threshold rule
4. Artifact versioned, checksummed, reloadable; a corrupt artifact is refused
5. Service: liveness, readiness, metrics, correlation ID, versioned prediction

16 passed

what `make session02` prints when you are done

Questions worth asking

- Northwind's Enterprise P0 SLA is 15 minutes. What recall target does that imply, and what precision can the on-call team absorb?
- The threshold was chosen on the validation split. What happens when next quarter's tickets look different?
- What in this service would you have to change if the model became a transformer? What would stay?